

BuildLang: Accountable Compute via Typed Capability Effects and Re-Derivable Receipts

Zain Dana Harper

Seattle, WA | [ORCID 0009-0001-7175-5393](https://orcid.org/0009-0001-7175-5393)

<https://github.com/HarperZ9/buildlang>

July 2026

Abstract

A program touches the outside world when it reads a file, opens a network socket, or asks the operating system for an environment value. This boundary is the natural unit of accountability for AI-assisted code and for scientific code. In most languages the boundary is hidden. It sits behind runtime helpers, macros, and foreign calls that never appear in a function’s type. BuildLang changes this. BuildLang is a compiler, written in Rust, for typed algebraic effects. An effect is a note in a function’s type that names an action the function may take on the outside world. In BuildLang, ambient capability access must appear in the function type. Ambient capability access means reaching the outside world without being handed the permission as an argument. Reading a file, opening a socket, or calling foreign code each requires a declared capability effect. There are eight such capabilities (FILESYSTEM, NETWORK, FOREIGN, and five others). The compiler tracks each capability conservatively through callbacks, closures, struct fields, async blocks, and control-flow selectors, so policy cannot be bypassed by hiding an effect behind an untyped function value. We use this effect system as a *witness mechanism*, a source of checked facts, for two families of re-derivable receipts. A receipt is a record that an independent party can check again later. The first is a codegen receipt that seals the source and the declared and observed capability facts. The second is a scientific-runtime receipt that compiles and runs a numeric kernel, checks a stated conservation or boundedness invariant over the captured series, and is verified by *re-running the program*. The scientific receipt ships seven mathematically distinct invariants: energy-monotone, conservation, the discrete maximum principle, an energy-balance residual, cross-column relation, approximate symplectic conservation, and a non-negative complexity bound. Each invariant has a paired negative-fixture kernel, a program built to fail on purpose, which demonstrates that the checker rejects a crafted violation. This is a bound on false negatives for those cases, not a completeness proof. Each scientific receipt also carries a mandatory overclaim boundary (NOT_A_NEW_PHYSICAL_LAW), so a receipt cannot be mistaken for a physical result it does not assert. We report 940 library tests passing with none failing, and a corpus of positive and negative kernel pairs across physics, quantum, chemistry, and algorithmic-complexity domains. The artifact is public. The receipts are designed so that an independent replayer re-derives the verdict without trusting stored bytes.

In plain terms

Imagine you hire someone to do a job inside your house. You want to know one thing before they start: which rooms will they enter, and will they carry anything out? A normal work order does not tell you. It says “fix the sink.” It does not say that the worker will also open your filing cabinet, or make a call from your phone, or carry a box to their van. To find out, you would have to follow the worker through the whole job and watch every step.

Software has the same problem. When a program runs, it can read your files, send data over the network, or run other code you cannot see. The description of the program, its type, usually does not tell you which of these it does. A reviewer has to read the whole program, and everything the program calls, to find out. The one fact that matters most for safety, what the program is allowed to touch, is the fact that is missing from its description.

BuildLang is a programming language that fixes this. In BuildLang, the description of a function must list every kind of outside action it takes. If a function reads a file, its type must say so. If it opens a network connection, its type must say so. If it calls unknown outside code, its type must say so. The language calls these outside actions capabilities. There are eight of them. If a function does one of these things without listing it, the program will not compile. The compiler even follows the action when it is passed around indirectly, for example when one function is handed to another as a value, so the fact cannot be hidden.

That gives us a trustworthy list of what a program touches. BuildLang then turns that list into a receipt. A receipt is a small record you can check. One kind of receipt records the source of the program and the list of things it touches, so an automatic gate can approve or refuse it. A second kind of receipt is for scientific programs. Suppose a program computes the motion of a physical system, and a quantity such as energy should stay constant. The receipt records that the program's output kept that quantity constant.

The important part is how you check a receipt. You do not trust the numbers written on it. You run the program again and check the answer again. If the program was tampered with, or if the quantity was not really kept constant, the re-run says so. Each receipt also states, in plain machine-readable form, what it does not claim. A scientific receipt never claims to have discovered a law of physics. It claims only that this program, on this run, kept the stated quantity in bounds. This paper is about how the language makes that list honest, how the receipts are built, and where their guarantees stop.

1 Introduction

Plainly: the most safety-relevant fact about a program, what it may touch, is the one fact left out of its type; BuildLang puts it in and uses it to build receipts you can re-check.

The boundary a program crosses when it touches the world is where accountability lives. A network call can send data out. A file read can pull in an unlogged dataset. A foreign call can hide any behavior behind an untyped symbol. Mainstream practice treats these as implementation details. The function signature says nothing about whether the body reads a file. The compiler does not track it. A reviewer, or a downstream policy gate, has to recover the fact by reading the whole transitive body. So the most security-relevant fact about a piece of code, what it is actually allowed to touch, is the one fact not in its type.

BuildLang inverts this. It is a statically-typed language with Hindley–Milner inference and *typed algebraic effects*. In this language the ambient capabilities a function may exercise are part of its type and are checked by the compiler. Ambient means the function reaches the outside world on its own, rather than being handed the access as an argument. A function that calls `read_file` must declare the `FILESYSTEM` effect. One that opens a socket must declare `NETWORK`. One that calls into a C function must declare `FOREIGN`. The same rule applies to compile-time macros (`include_str!`, `env!`, `println!`) and to known C-runtime helper aliases, so ambient access cannot hide behind a macro expansion or a renamed helper. The capability provenance is tracked through the language's value channels. An effectful callback passed into a wrapper, stored in a struct field, selected by an `if` or `match`, captured by a closure, or awaited as a future all keep their declared capability row. So a policy gate sees which boundary function a higher-level workflow actually depends on.

The effect system is not the deliverable. It is the *witness mechanism* for two receipt families that are. A *codegen receipt* seals the entry source, the resolved module graph, and the declared,

observed, and propagated capability facts into a deterministic artifact a CI gate can re-check. A *scientific-runtime receipt* compiles and runs a numeric kernel, captures its numeric output as a measurement series, and checks a stated invariant. The invariant might be that energy is non-increasing, that a quantity is conserved, that a residual stays at roundoff, or that a complexity slack stays non-negative. The receipt seals the verdict. Verification re-runs the program and re-checks the verdict. It deliberately tolerates platform float differences by comparing the verdict rather than exact bits.

The paper makes four contributions:

1. **Capability effects with provenance through values** (Section 3): eight ambient capabilities surfaced in function types, tracked conservatively through the value channels where effects are usually lost (callbacks, closures, fields, selectors, futures).
2. **Accountable codegen receipts** (Section 4): a deterministic receipt and a policy-profile language that turns observed capability evidence into an enforceable CI gate.
3. **Accountable scientific compute** (Section 5): seven invariants over a captured numeric series, each with a paired negative fixture, verified by re-execution, with effect-derived dataset and determinism fields and a mandatory overclaim boundary.
4. **An honesty discipline** (Section 6): every receipt states what it does *not* claim, in machine-readable form; the verifier is shown to be able to fail via negative fixtures and a self-test.

BuildLang is the systems instance of a broader idea pursued in a companion artifact, EMET [1]: witnessed, re-derivable receipts with closed verdict lattices and mandatory failure-proofs. EMET applies the idea to byte integrity. BuildLang applies it to codegen facts and numeric invariants, with the typed-effect system as the witness that supplies the facts.

2 Background and related work

Plainly: BuildLang builds on algebraic effects and capability security, but treats the effect row as sealed evidence, and its scientific receipt wraps standard numerical checks in accountability rather than inventing a new method.

Algebraic effects. Algebraic effects and handlers [2, 3] separate the *performance* of an effect from its *interpretation*. This gives compile-time effect tracking, with caller-controlled handlers and no runtime overhead beyond a continuation jump. BuildLang adopts this model. Effects are declared with `~` and handled with `handle` and `with`, and the C backend compiles effects to `setjmp` and `longjmp`. BuildLang departs from the effects-language line in two ways. It treats *ambient* capabilities as first-class effect requirements rather than user-defined operations. And it uses the effect row as sealed evidence, not only as a type-system guarantee.

Capability security. Object-capability languages restrict authority to what is reachable. BuildLang’s capability effects are the static analogue, lifted into the type so that capability use is visible at every call site rather than inferred from reachability. The closest contrast is with two other approaches. One is languages that track effects but do not bind them to enforceable receipts. The other is security linters that recover capability use by post-hoc analysis rather than by construction.

Reproducible and verified scientific compute. Conserved-quantity and boundedness checks are standard diagnostics in numerical analysis. Examples are discrete maximum principles, energy estimates for parabolic schemes, and the near-conservation of energy by symplectic integrators. BuildLang’s scientific receipt is not a new numerical method. It is an *accountability wrapper*. It seals the fact that a compiled kernel’s observed series satisfies a stated such property, verified by re-running the program, with negative fixtures proving the check is not vacuous. The funnel-hashing probe bound we reuse as an algorithmic-complexity example is from prior work [4].

Reproducible builds. The codegen receipt shares the reproducible-builds [5] instinct, which is to seal the source and re-derive the artifact. But it seals the *capability facts and verdict* rather than requiring byte-identical artifacts. That is an explicit non-goal we defend in Section 5.

3 Capability effects

Plainly: eight kinds of outside action must be written in a function’s type, and the compiler keeps tracking each one even when the action is passed around as a value.

3.1 Eight ambient capabilities

A function that exercises an ambient capability must declare it. The eight built-in capabilities and their direct surfaces are:

Capability	Direct surfaces
FILESYSTEM	<code>read_file</code> , <code>write_file</code> , <code>file_exists</code> , <code>list_dir</code> , compile-time <code>include!/include_str!/include_bytes!</code>
NETWORK	<code>tcp_connect</code> , <code>tcp_send</code> , <code>tcp_recv</code> , <code>tcp_close</code>
PROCESS	<code>exit</code> , <code>process_exit</code>
ENVIRONMENT	<code>getenv</code> , <code>args_get</code> , compile-time <code>env!/option_env!</code>
CLOCK	<code>clock_ms</code> , <code>time_unix</code>
CONSOLE	<code>read_line</code> , <code>println!/print!/eprintln!</code> , diagnostic logging
FOREIGN	calls to unknown functions in <code>extern</code> blocks; reads from foreign statics
GPU	direct <code>build_vk_*</code> and <code>build_gfx_*</code> runtime helpers

Known C-runtime helper aliases declared through `extern` blocks are classified by their specific capability. For example, `build_read_file` is FILESYSTEM and `build_gfx_init` is GPU. Only genuinely unknown extern functions remain FOREIGN. Compile-time macros are direct capability surfaces: `println!(read_file("ops.toml"))` requires both CONSOLE and FILESYSTEM, and the receipt records both. The compiler rejects a function that performs a capability without declaring it, with a diagnostic naming the ambient call or macro.

3.2 Provenance through value channels

The hard part is not declaring effects on direct calls. It is retaining capability provenance through the value channels where effects are conventionally lost. BuildLang tracks the following escape classes, conservatively, and each is backed by regression tests:

- **First-class function values.** An effectful callback type (`fn()` with `FileSystem`) carries its effect row. A pure `fn()` parameter cannot accept `read_file`, and a cast to a pure callback type is rejected. Aliases (`let loader = read_file`) and closures inherit the capability at the call site.
- **Aggregate storage.** Effectful callbacks stored in struct fields, tuple slots, enum variants (including struct-like variants), and indexed tables retain their origins. Calls through `ops.loader`, `loaders.0`, or `loaders[0]` record the stored callee as propagated evidence.
- **Control-flow selection.** A function selected by `if`, `if let`, or `match` records every possible branch target. Conditional and loop assignment merge the possible exits rather than keeping only the last path visited.
- **Async.** An `async` block is a delayed effect value. Its construction is pure, and `await` inherits the body’s capability row, recording both the awaited expression and the latent origin.

- **Destructuring and update.** Tuple, struct, enum-variant, and slice destructuring, and struct-update expressions, refresh access paths without forcing policies to allow stale construction aliases.

The unifying rule is that *operational access has to appear in the function type instead of hiding behind a runtime helper, macro expansion, dynamic dispatch, or an untyped function value*. Trait-object method calls are checked from the trait signature, so `loader.load()` through `dyn Loader` records `Loader.load` rather than disappearing behind dispatch.

4 Accountable codegen receipts

Plainly: one command writes a checkable record of a program’s source and what it touches, and another command re-checks it against a stated policy, so a CI gate approves code by evidence rather than trust.

`buildc check <file> --receipt <path>` writes a deterministic `buildlang-check-receipt/v1` JSON artifact. The artifact holds compiler and language version metadata, a SHA-256 digest of the entry source bytes, an `input_digests` ledger for every entry, import, include, and module file read by the check pipeline, an `input_graph_digest` fingerprint for the whole checked graph, the declared effects, the observed capability sources (`observed_capabilities`), the effectful callees a caller inherits (`propagated_effects`), pass or fail status, and compact diagnostics.

`buildc receipt verify` re-checks a saved receipt against the current source graph. It verifies the receipt schema and the compiler and language identity, re-derives the entry source and input-graph digests, replays the compiler check, and compares the saved `declared_effects`, `observed_capabilities`, `propagated_effects`, diagnostics, and policy violations against the current result. Verification can be pinned to a built-in profile (`--expect-profile ci-review`) or to an exact policy digest (`--expect-policy-digest`). So a CI gate proves the receipt was accepted under a specific policy rather than whichever policy it happens to carry.

Policy profiles. A `buildlang-check-policy/v1` document turns receipt evidence into a gate. It has `allowed_effects` (denied effects always fail); `direct_effect_allowlist` and `direct_capability_source_allowlist`, which narrow approved direct boundaries to specific helpers, macros, or FFI sources; `propagated_effect_allowlist` and `propagated_effect_source_allowlist` which narrow approved propagated callers to specific callees; and coverage requirements that reject stale allowlist entries. Built-in profiles range from `pure` (denies every ambient capability), through `console-only`, `offline` (local file, environment, clock, and console, with no network, process, FFI, or GPU), and `ci-review`, to `strict-accountability`. The policy is evaluated against the effect-system facts, not against assertions.

5 Accountable scientific compute

Plainly: for a numeric program, the receipt records that a quantity which should stay bounded actually stayed bounded, and you check it by running the program again.

5.1 The receipt

`buildc run --emit-receipt` compiles a program to C and runs it. It captures every whitespace- or newline-separated token that parses as a finite `f64` as one entry of a measurement series, checks a stated invariant over that series, and emits a sealed `buildlang-scientific-runtime-receipt/v0`. Non-finite values mean divergence. Parsing stops at the offending token, only the finite prefix is stored, and the receipt is `UNVERIFIABLE` with a `NONFINITE_OBSERVED` label. A diverged run is never a `PASS`.

5.2 The invariant family

Each invariant reduces the series to a *violation count*. The verdict is uniform across the family: PASS if and only if the violation count is zero and the series has at least two points. The tolerance is a fixed property of the invariant, sealed in the receipt. The verifier rejects a receipt that resealed a non-canonical tolerance, so a receipt cannot weaken its own check. The seven invariants are mathematically distinct: the same series yields different verdicts across them, which is why they are separate.

Invariant	sealed name	a step violates when	tolerance
energy-monotone	energy_monotone_nonincreasing	$s[k+1] > s[k] + tol$	10^{-12}
conservation	conserved_quantity_constant	$ s[k] - s[0] > tol$	10^{-9}
bounded	bounded_by_initial_maximum	$s[k] > s[0] + tol$	10^{-9}
energy-identity	energy_identity_residual	$ s[k] > tol$	10^{-9}
relation	relation_columns_agree	row columns differ by $> tol$	10^{-9}
conserved-band	conserved_within_band	$ s[k] - s[0] > tol$	5×10^{-3}
non-negative	non_negative	$s[k] < -tol$	10^{-9}

Conservation fences both sides of $s[0]$. Bounded (the discrete maximum principle) fences only the upper side, so a series that dips and returns to its initial value passes bounded while failing conservation. Energy-monotone forbids any step-wise rise. Energy-identity is the odd one out: its reference is zero (an absolute bound), so every step including step 0 is checked, and its series is a per-step energy-balance residual rather than a trajectory. Conserved-band is approximate conservation. A leapfrog oscillator’s energy oscillates in a measured $\sim 1.25 \times 10^{-3}$ band around H_0 with no secular drift, passing the 5×10^{-3} budget, while explicit Euler’s energy grows by $(1+dt^2)$ per step and leaves the band within two steps. Non-negative is the first algorithmic member. A binary search on 1024 elements prints the slack 11 – probes per lookup, which stays non-negative (pass), whereas a linear scan’s slack drops to about -1013 (fail).

5.3 Negative fixtures

Every invariant ships a paired positive and negative kernel. A negative fixture is a program whose invariant is *expected* to fail. Run with `--negative-fixture` it yields `FAIL_EXPECTED`, which demonstrates that the checker catches these specific crafted violations. This is evidence against false negatives on the fixture set, not a proof of completeness. Without the flag the same failing run is `FAIL_UNEXPECTED`, because an unexpected invariant violation is a red flag, not a demonstration of the checker. The corpus spans four domains. In physics it uses the 1-D heat equation FTCS energy dissipation, and the symplectic versus explicit-Euler oscillator. In quantum it uses Born-rule normalization under a unitary gate versus a non-unitary gain. In chemistry it uses a reversible-reaction atom balance versus a stoichiometry bug. In algorithms it uses binary-search and funnel-hashing [4] probe bounds versus linear scans.

5.4 Verify by re-running

`buildc receipt verify` dispatches on the receipt schema. For a scientific-runtime receipt it *re-runs and re-checks* rather than trusting stored numbers. It recomputes the seal, which is an integrity gate before any sealed field is interpreted. It re-derives the source and input-graph digests, re-runs the program with the recorded arguments, re-checks the measurement count, and recomputes the verdict triple (status, violation count, receipt status). Any drift is a typed verification failure. It checks the *verdict*, not exact floats, so a receipt emitted on one machine re-verifies on another despite last-bit differences. The verdict-count rule is robust to platform float variation and codegen reassociation. Diverged runs are the exception. The finite-prefix

length is platform-dependent, so when both the receipt and the re-run record divergence, the reproduced divergence itself is the faithfulness signal.

A receipt is faithful if it reproduces, but a faithful receipt that records an unexpected failure is not a pass. The exit code reflects the verdict too. So `buildc receipt verify r.json && deploy` is safe. It will not deploy on a receipt that records an unexpected invariant violation or a diverged run, while a negative fixture reproducing its expected failure legitimately passes.

5.5 Effect-derived honesty fields

The typed-effect system supplies three fields of the scientific receipt as *witnessed facts*. They are derived fail-closed from the capability set rather than asserted. The first is `input_dataset`. A capability is a dataset hazard unless it provably cannot feed external data. `FILESYSTEM`, `NETWORK`, `ENVIRONMENT`, `FOREIGN`, `GPU`, and `stdin-reading CONSOLE` all fence this field as `POSSIBLE_UNWITNESSED`; a program with none of them provably consumed no external dataset. The second is `seed`, which is not applicable in `v0`, since the language has no RNG builtin. The third is `determinism`, derived from every nondeterminism source the language exposes. Any capability this build does not recognize is treated as a hazard, so a capability added later cannot silently widen the claim.

6 The honesty discipline

Plainly: every receipt says in machine-readable form what it does not claim, and the verifier itself is proven able to fail, so a passing check means something.

Every scientific receipt carries the label `NOT_A_NEW_PHYSICAL_LAW` and a machine-readable `not_claimed` set that *must* include `physical_law`, or `verify` rejects the receipt outright. The receipt witnesses one thing: the compiled program’s observed output series satisfies the stated invariant, or, for a negative fixture, expectedly violates it. It does not prove that the underlying PDE is solved correctly, that the discretization is convergent, that the parameters are physical, or that the result matches a reference solution. Those are the program author’s responsibility, and the receipt makes no statement about them.

The discipline is symmetric: a verifier worth trusting must be shown to fail. Negative-fixture kernels close the can-it-fail gap on the invariants. `buildc receipt verify --self-test` closes it on the verifier, by tampering with several distinct sealed fields and asserting that each is rejected with its expected typed failure class. The failure taxonomy is closed and stable: `MALFORMED`, `SCHEMA_UNSUPPORTED`, `COMPILER_MISMATCH`, `OVERCLAIM_BOUNDARY_MISSING`, `FIELD_CONTRACT_VIOLATION`, `EFFECT_POLICY_DRIFT`, `SOURCE_DIGEST_MISMATCH`, `MEASUREMENT_COUNT_DRIFT`, `INVARIANT_STATUS_DRIFT`, `SEAL_MISMATCH`, and `INVARIANT_NOT_HELD`, each with a stable exit code. Receipts are parsed through a strict reader that rejects duplicate object keys at any depth. With a permissive last-duplicate-wins reader, a document carrying two `receipt_status` keys can show one value to a hasher and another to a reader, a seal-forgery vector.

A receipt chain binds multi-stage computations into one ordered, tamper-evident bundle. `chain build` records each member’s seal in order and computes a chain seal. `chain verify` recomputes the chain seal, checks each member’s current seal against the pinned seal, and re-verifies each member through the real `verify` path. A `corpus` command runs every kernel pair, asserts the emitted status equals the declared one, and re-verifies each receipt. It is a gate that can fail rather than a rubber stamp.

Seals are not byte-reproducible, by design. The scientific receipt seals the SHA-256 of the compiled program binary, and that binary is not reproducible across builds. The C toolchain embeds timestamps, temp paths, and link order. `Verify` never re-checks the executable digest

for equality. It re-runs the program and re-checks the verdict, and executable reproduction is *reported*, never required. Byte-reproducibility of the receipt artifact is a non-goal. Re-checkability of the verdict is the guarantee.

7 Evaluation

Plainly: the compiler passes its test suites, the ten kernel pairs each show a real pass and a real fail, and every escape route for a hidden effect has a test that closes it.

The compiler passes 940 library tests, 140 binary, 309 CLI, 52 lexer, and 88 parser tests with none failing (baseline 2026-07-02), with `cargo fmt --check` clean. The library tests cover type inference and effects; the experimental `#[linear]` no-cloning attribute, which conservatively rejects a regression-tested set of compositional escapes but is honestly marked not yet fully sound; lexer and parser units; the MIR and codegen paths across backends; the semantic-corpus receipt builders; and LSP dispatch.

The scientific-compute evaluation is the ten positive and negative kernel pairs of Section 5.3. Each pair is a falsifiable demonstration. The positive kernel passes its invariant, the negative kernel fails it, and the corpus command asserts both classifications hold and both receipts re-verify. The capability-provenance evaluation is the regression-tested coverage of the escape classes of Section 3.2. Each escape class (callback alias, struct-field storage, control-flow selection, async, destructuring, and the rest) has a test that a pure context cannot silently absorb an effectful value.

8 Discussion

Plainly: both receipt families rest on one mechanism, the typed-effect witness, and the open problems are stated plainly rather than hidden.

BuildLang’s two receipt families share a single mechanism: the typed-effect system witnesses the facts a receipt seals. The codegen receipt seals capability facts directly. The scientific receipt derives its dataset, seed, and determinism fields from the same capability facts, fail-closed. This is the systems analogue of the companion EMET witness [1]. Both seal a re-derivable verdict rather than trusting stored bytes. Both carry a closed verdict lattice. Both insist on a failure-proof. And both bound their own claims in machine-readable form. The difference is the witness source: SHA-256 over raw bytes for EMET, and the type-and-effect system for BuildLang.

The open questions are honest. The `#[linear]` no-cloning attribute is not yet fully sound; full soundness needs an affine or borrow checker on the MIR. The scientific receipt checks one invariant over one series per receipt (v0). Cross-language parity of the experimental stripped-credential rebind is a follow-on. The receipt seal is an unkeyed SHA-256 over the canonical form. It detects accidental corruption, but anyone can recompute it after editing descriptive metadata. Integrity for the verdict that matters comes from the re-run, not from the seal. We state this rather than overclaim cryptographic tamper-proofing.

9 Reproducibility

Plainly: everything described here is public, and a reader can re-make any receipt and re-check it by re-running the program.

BuildLang, its compiler, the eight capability effects, the codegen and scientific-runtime receipts, and the ten positive and negative scientific kernel pairs are public under the project license at <https://github.com/HarperZ9/buildlang>. A reader can re-emit any receipt from its source and re-verify it by re-running the program, without trusting the stored series byte-for-byte.

Availability

Plainly: the source is public at the link below. BuildLang is at github.com/HarperZ9/buildlang.

References

- [1] Harper, Z. D. *EMET: An Authority-Incapable Byte-Level Integrity Witness for AI Oversight*. Companion paper, 2026.
- [2] Bauer, A., and Pretnar, M. *Programming with Algebraic Effects and Handlers*. J. Log. Algebr. Meth. Program. 84(1):108–123, 2015. doi:10.1016/j.jlamp.2014.02.001.
- [3] Plotkin, G. D., and Pretnar, M. *Handlers of Algebraic Effects*. ESOP, 2009.
- [4] Farach-Colton, M., Krapivin, A., and Kuszmaul, W. *Optimal Bounds for Open Addressing Without Reordering*. arXiv:2501.02305, 2025.
- [5] Reproducible Builds Project. *Reproducible Builds: a set of software development practices*. <https://reproducible-builds.org>, 2026.
- [6] NIST. *Secure Hash Standard (SHS)*. FIPS PUB 180-4, 2015.